

- Spring boot란?

: 스프링 프레임워크 기반 프로젝트를 복잡한 설정 없이 쉽고 빠르게 만들어주는 라이브러리

: 사용자가 일일이 모든 설정을 하지 않아도 자주 사용되는 기본설정을 알아서 해줌.

- 왜 사용?

:스프링 프레임워크를 사용하려면 많은 XML 설정 파일(web.xml, rootContext.xml, ServletContext.xml 등)들을 작성 해 야하고, 설정 방법을 모두 외우지 못했다면 기존에 사용했던 설정을 Copy&Paste하거나 개발자가 일일이 인터넷 검색을 통해서 설정해주어야 하는데 이는 곧 생산성과 비용 문제와 연결된다.

하지만

스프링 부트를 사용하면 복잡한 설정 없이 쉽고 빠르게 스프링 프레임워크를 사용할 수 있다.

- 장점

1) 라이브러리 관리의 자동화

Spring boot 버전을 선택하면 그 버전 안에서 관련된 라이브러리들이 가장 호환성이 좋은 라이브러리를 관리해준다.스프링 부트의 Starter 라이브러리를 등록해서 라이브러리의 의존성을 간단하게 관리할 수 있다

2) 라이브러리 버전 자동 관리

기존에는 스프링 라이브러리의 버전을 하나하나 직접 입력해야 했지만, 스프링 부트는 pom.xml에 스프링 부트 버전을 입력하면 스프링 라이브러리 뿐만 아니라 서드 파티 라이브러리 들도 호환되는 버전으로 알아서 다운해준다

3) 설정의 자동화

스프링 부트는 @EnableAutoConfiguration 어노테이션을 선언해서 스프링에서 자주 사용 했던 설정들을 알아서 등록해준다.

4) 내장 Tomcat

스프링 부트는 Tomcat을 내장하고 있기 때문에 @SpringBootApplication 어노테이션이 선언되어있는 클래스의 main() 메소드를 실행하는 것만으로 서버를 구동시킨다. 내장 톰캣을 사용하려면 특별한 설정 없이 Web Starter 의존성만 추가해주면 된다.

5) 독립적으로 실행 가능한 JAR

웹 프로젝트라면 WAR 파일로 패키징 해야하지만 스프링 부트는 내장 톰캣을 지원하기 때문에 JAR 파일로 패키징해서 웹 애플리케이션을 실행시킬 수 있다.

spring boot의 기본설정문서

- 1) application.properties -> key = value 설정
- 2) application.yml -> json형태 처럼 key:value 설정

spring boot에서 정적문서 위치

src/main/resources/static 에 html, css, img, js 문서를 넣는다.

spring boot는 기본적으로 view template(react, vue.js, thymleaf) 이 jsp로 설정되어 있지 않아서

jsp를 사용하려면 별도의 설정이 필요하다. (lib 등록 , InternalResourceViewResolver 등록)

- 1) pom.xml에 추가

```
<!-- jsp view를 사용하기 위한 설정 -->
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
</dependency>

<dependency>
  <groupId>jakarta.servlet</groupId>
```

```
<artifactId>jakarta.servlet-api</artifactId>
<version>6.0.0</version>
<scope>provided</scope>
</dependency>

<dependency>
  <groupId>jakarta.servlet.jsp.jstl</groupId>
  <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>

</dependency>

<dependency>
  <groupId>org.glassfish.web</groupId>
  <artifactId>jakarta.servlet.jsp.jstl</artifactId>

</dependency>
```

2) application.proerties에 설정

```
server.port=9000

#jsp view
spring.mvc.view.prefix=/WEB-INF/views/
spring.mvc.view.suffix=.jsp
```

IntelliJ에 Springboot 프로젝트 만들기

New Project

Search

New Project
Empty Project

Generators

- Maven Archetype
- Jakarta EE
- Spring Initializr
- JavaFX
- Quarkus
- Micronaut
- Ktor
- Compose for Desktop
- HTML
- React
- Express
- Angular CLI
- Vue.js
- Vite

Server URL: start.spring.io

Name:

Location: 
Project will be created in: ~\Downloads\mvc\step01_bootJPA

☐ Create Git repository

Language: ☐ Java ☐ Kotlin ☐ Groovy

Type: ☐ Gradle - Groovy ☐ Gradle - Kotlin ☐ Maven

Group:

Artifact:

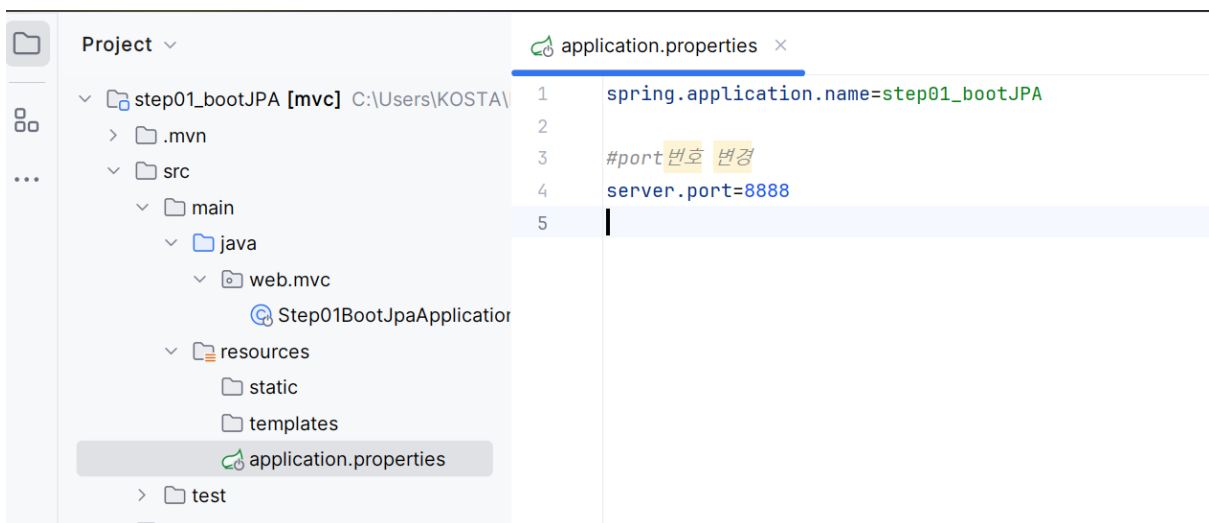
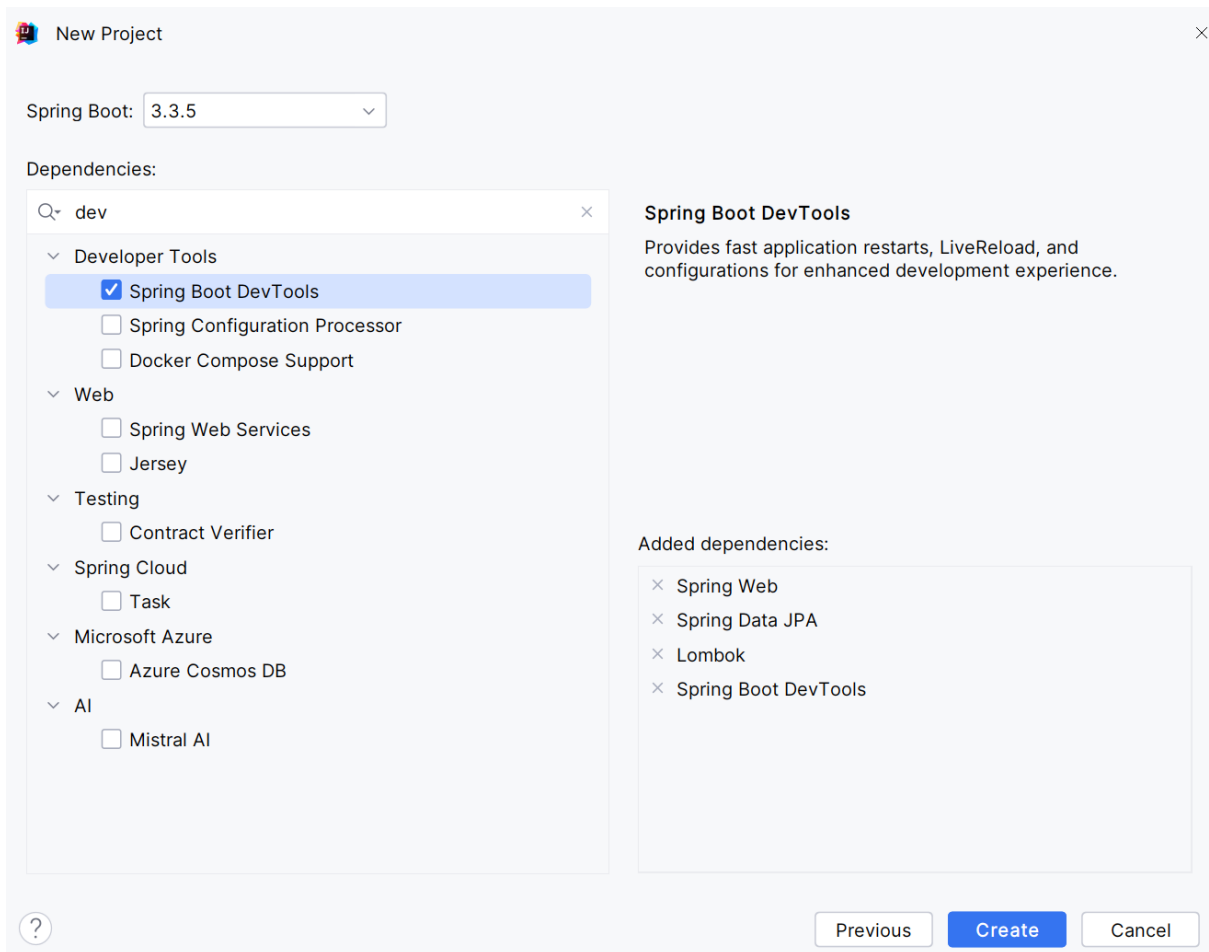
Package name:

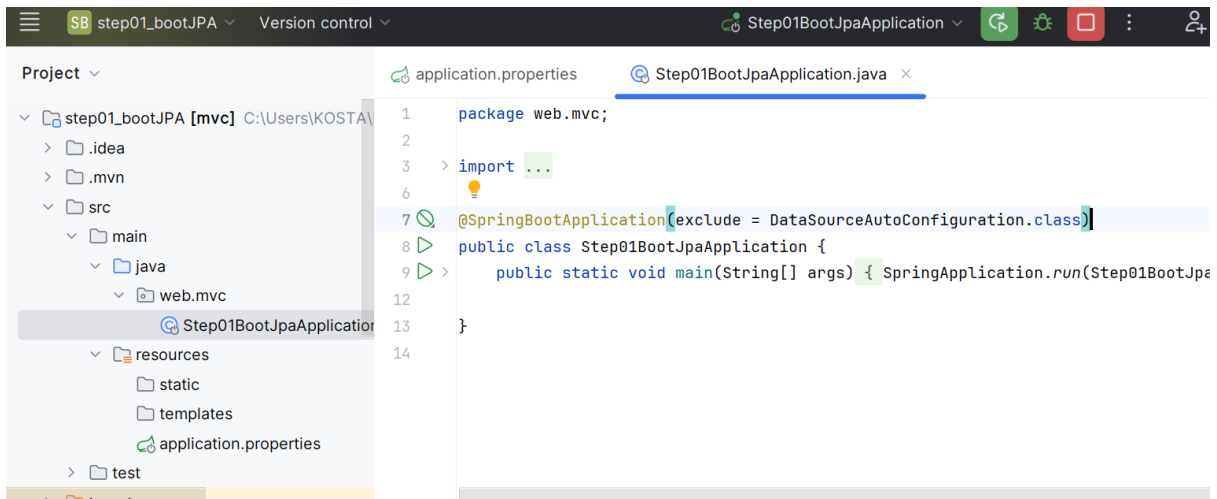
JDK:

Java:

Packaging: ☐ Jar ☐ War

 Next Cancel





```

[ restartedMain] web.mvc.Step01BootJpaApplication      : Starting Step01BootJpaApplication using Java 17.0.9 with PID 1
[ restartedMain] web.mvc.Step01BootJpaApplication      : No active profile set, falling back to 1 default profile: "def
[ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer            : Tomcat initialized with port 8888 (http)
[ restartedMain] o.apache.catalina.core.StandardService              : Starting service [Tomcat]
[ restartedMain] o.apache.catalina.core.StandardEngine               : Starting Servlet engine: [Apache Tomcat/10.1.31]
[ restartedMain] o.a.c.c.C.[Tomcat].[localhost].[/]                  : Initializing Spring embedded WebApplicationContext
[ restartedMain] w.s.c.ServletWebServerApplicationContext           : Root WebApplicationContext: initialization completed in 567 ms
[ restartedMain] o.s.b.d.a.OptionalLiveReloadServer                  : LiveReload server is running on port 35729
[ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer            : Tomcat started on port 8888 (http) with context path '/'
[ restartedMain] web.mvc.Step01BootJpaApplication                    : Started Step01BootJpaApplication in 0.725 seconds (process run
[ restartedMain] .ConditionEvaluationDeltaLoggingListener             : Condition evaluation unchanged

```

[Spring Boot Application 에 설정된 어노테이션 설명]

@SpringBootApplication

@EnableJpaRepositories(basePackages = {"com.example.springedu.repository"})

@EntityScan(basePackages = {" com.example.springedu.entity"})

- @SpringBootApplication 어노테이션은 스프링 부트의 가장 기본적인 설정을 선언한다. 내부적으로 @ComponentScan과 @EnableAutoConfiguration을 설정한다.

- @ComponentScan 어노테이션은 스프링 3.1부터 도입된 어노테이션이며 스캔 위치를 설정한다. (프로젝트 생성시 기본으로 만들어지는 패키지의 하위에 클래스들을 두는 경우에는 생략 가능)

- @EnableJpaRepositories 어노테이션은 JPA Repository들을 활성화하기 위한 어노테이션이다.

- @EntityScan 어노테이션은 엔티티 클래스를 스캔할 곳을 지정하는데 사용한다.

[Spring Boot 테스트]

스프링 부트는 서블릿 기반의 웹 개발을 위한 spring-boot-starter-web, 유효성 검증을 위한 spring-boot-starter-validation 등 spring-boot-starter 의존성을 제공하고 있다. 테스트를 위한 spring-boot-starter-test 역시 존재하는데, 다음과 같은 라이브러리들이 포함된다.

- 1) JUnit 5: 자바 애플리케이션의 단위 테스트를 위한 사실상의 표준 테스트 도구
- 2) Spring Test & Spring Boot Test: 스프링 부트 애플리케이션에 대한 유틸리티 및 통합 테스트 지원
- 3) AssertJ: 유연한 검증 라이브러리
- 4) Hamcrest: 객체 Matcher를 위한 라이브러리
- 5) Mockito: 자바 모킹 프레임워크
- 6) JSONassert: JSON 검증을 위한 도구
- 7) JsonPath: JSON용 XPath

spring-boot-starter-test 의존성을 추가하면 스프링이 미리 만들어둔 테스트를 위한 다양한 어노테이션을 사용해 편리하게 테스트를 작성할 수 있다.

```

@SpringBootTest
@WebMvcTest
@DataJpaTest
@RestClientTest
@JsonTest
@JdbcTest
.....
[ @DataJpaTest 애노테이션 ]

@DataJpaTest

```

JPA 레포지토리 테스트를 위해서는 @DataJpaTest를 이용할 수 있다.

@DataJpaTest는 기본적으로 @Entity가 있는 엔티티 클래스들을 스캔하며 테스트를 위한 TestEntityManager를 사용해 JPA 레포지토리들을 설정해준다.

스프링은 테스트에 @Transactional이 있으면 테스트가 끝난 후 자동으로 트랜잭션을 롤백한다.

@DataJpaTest에는 @Transactional 어노테이션이 포함되어 있어서 기본적으로 모든 테스트가 롤백된다. 만약 롤백을 원하지 않는다면 @Rollback(false)를 추가하면 된다.

```

@DataJpaTest
@Rollback(false)
class MyRepositoryTests {

    // ...

}

```

또한 만약 H2와 같은 내장 데이터베이스가 클래스 패스에 존재한다면 내장 데이터베이스가 자동 구성된다. spring-boot-test 의존성에는 기본적으로 H2가 들어있으므로 별다른 설정을 주지 않는다면 H2로 설정된다. 내장 데이터베이스로 설정되기를 원하지 않는다면 다음과 같이 AutoConfigureTestDatabase의 replace 속성을 NONE으로 주면 된다.


```
@DataJpaTest
@AutoConfigureTestDatabase(replace = Replace.NONE)
class MyRepositoryTests {

    // ...

}
```

테스트 어노테이션	@SpringBootTest	@WebMvcTest	@DataJpaTest
테스트 종류	통합테스트	부분테스트 (슬라이스 테스트)	부분테스트 (슬라이스 테스트)
등록되는 빈	모든 빈	Controller와 연관된 빈	JPA Repository와 연관된 빈